

Yassine DEROUICH
Boubacar Loulou DIALLO
E5FR

Projet



Cloud Security

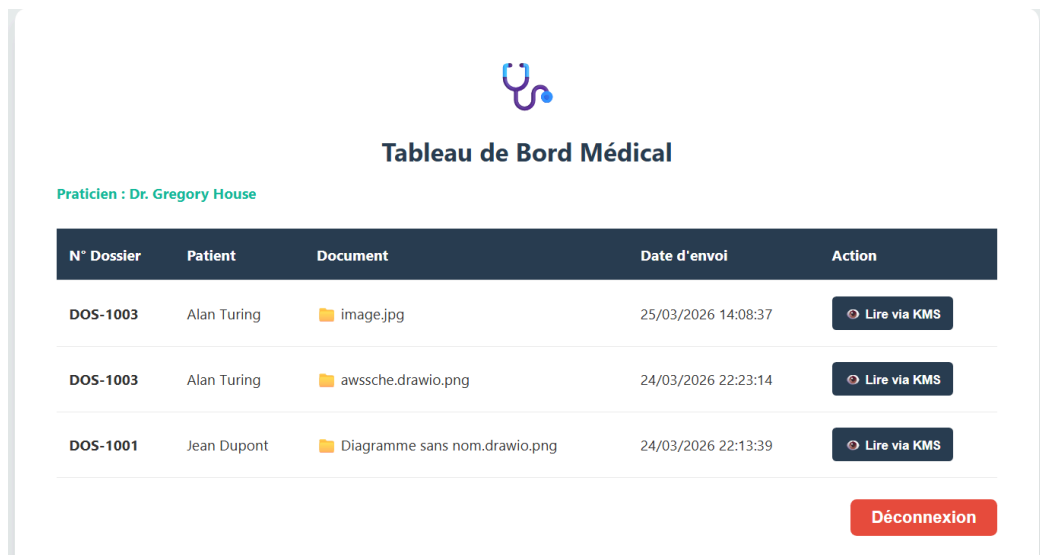
Medishare, l'application sécurisée de
partage de documents entre médecin et
patient.

Introduction	2
Phase 1-A : Configuration et gestion des buckets S3	3
Phase 1-B : Architecture et gestion des coûts	4
Phase 2 : Mise en œuvre de la sécurité dans les environnements AWS	7
Phase 3 : Gestion des clés (KMS) et Secrets Manager	13
Phase 4 : Surveillance et conformité	15
Phase 5 : Application de DevSecOps	17
Recherche : Attaques adverses en cybersécurité des plateformes de Gestion de Contenus Numériques	21
1. Identification et analyse des types d'attaques	21
A). Manipulation de contenus et Attaques IA	21
B). Attaques applicatives et réseau traditionnelles	22
2. Impacts potentiels sur la sécurité et la confidentialité	22
A) Impact sur la Confidentialité (Violation de données) :	22
B) Impact sur l'Intégrité (Mise en danger d'autrui) :	22
C) Impact sur la Disponibilité (Rupture de soin) :	23
3. Stratégies de défense et bonnes pratiques	23
A) Signatures numériques et Hachage :	23
B) Confidentialité différentielle :	23
C) Contre les Injections de code :	23
D) Contre le Déni de Service (DDoS) :	23
E) Bouclier Réseau :	24
Protection globale et DevSecOps :	24
Conclusion	24

Introduction

Ce projet consiste à concevoir et déployer une plateforme de gestion de contenus sécurisée dans le domaine de la santé (prototype). Il s'agit d'un site où les patients et les médecins peuvent consulter leurs documents partagés. Cette idée nous est venue après s'être rendu compte que certains rendez-vous médicaux planifiés plusieurs mois à l'avance pouvaient nous contraindre à perdre des documents papiers. Ainsi, une solution digitale ferait ce problème d'ordonnances/radios perdues.

Cette plateforme permet à un patient, avec un dossier unique, de partager à son médecin attiré des documents. Le médecin attiré se connecte alors sur la plateforme et peut consulter les fichiers du patient.



Le projet intègre des pratiques DevSecOps, utilise les services AWS pour l'infrastructure cloud, et inclut une recherche sur les attaques adverses en cybersécurité pertinentes au contexte de la gestion de contenus numériques.

Objectifs pédagogiques :

- Concevoir et déployer une application web sécurisée pour la gestion et le partage de contenus numériques.
- Intégrer des pratiques DevSecOps pour assurer la sécurité continue tout au long du cycle de développement.
- Utiliser les services AWS pour l'hébergement, le stockage, la gestion des clés et la surveillance de l'application.
- Réaliser une recherche sur les attaques adverses spécifiques aux plateformes de gestion de contenus numériques.
- Conceptualiser l'architecture cloud de la plateforme en utilisant Cloudcraft.

Phase 1-A : Configuration et gestion des buckets

S3

Objectif : Infrastructure de stockage immuable et auditable.

Réalisation : Création du bucket medishare-patient-data-scnvbi. Activation du chiffrement AES-256 et du versioning pour prévenir les suppressions accidentelles.

Dans le cadre de ce projet, nous avons créées plusieurs bucket s3.

Il y a le bucket s3 medishare-audit qui contient les logs, notamment utilisé par cloudtrail et pour faire les rapports d'inventaire s3.

Il y a ensuite le bucket s3 medishare-patient-data-scnvbi contenant les fichiers clients

General purpose buckets (3) [Info](#)   Copy ARN  Empty  Delete  Create bucket

Buckets are containers for data stored in S3.

	Name	AWS Region	Creation date
☐	medishare-audit-scnvbi	Europe (Ireland) eu-west-1	March 25, 2026, 10:10:48 (UTC+01:00)
☐	medishare-patient-data-scnvbi	Europe (Ireland) eu-west-1	March 24, 2026, 21:21:54 (UTC+01:00)

Le bucket des données utilisateurs est chiffré par kms

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [Info](#)

Server-side encryption with AWS Key Management Service keys (SSE-KMS)

Encryption key ARN

 [arn:aws:kms:eu-west-1:371447792143:key/2c8144a0-6b61-4cd6-b2ef-a14e1024a148](#) 

Bucket Key

When KMS encryption is used to encrypt new objects in this bucket, the bucket key reduces encryption costs by lowering calls to AWS KMS.
Enabled

Avec la clé créée via terraform:

```
resource "aws_kms_key" "medishare_kms_key" {
  description      = "Clé KMS pour chiffrer les données patients MediShare"
  deletion_window_in_days = 7    # Permet de détruire la clé rapidement si besoin
  enable_key_rotation = true # rotation automatique

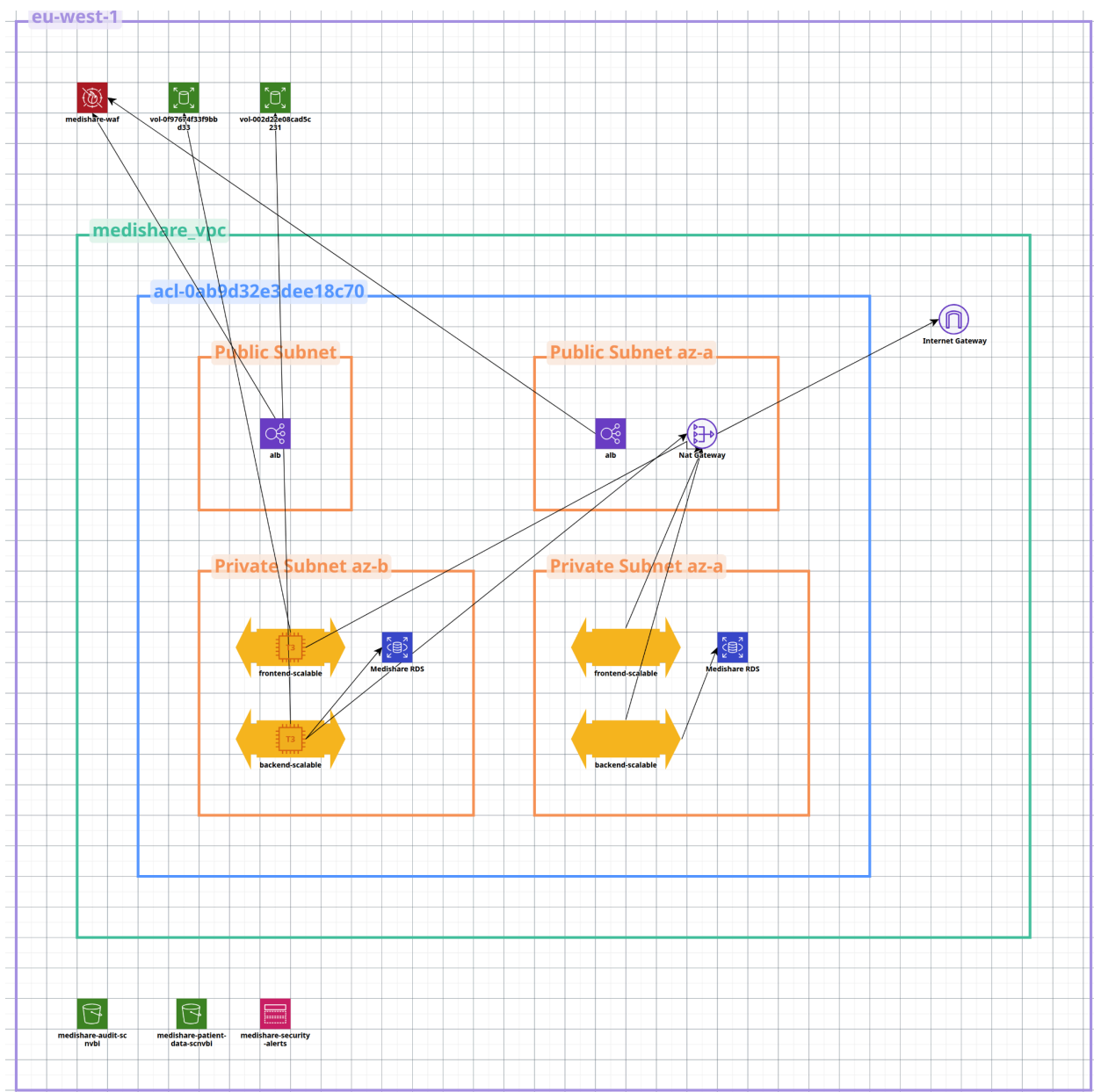
  tags = { Name = "medishare-kms-key" }
}
```

medishare-patient-data-scnvbi [Info](#)

Objects	Metadata	Properties	Permissions
Bucket overview AWS Region Europe (Ireland) eu-west-1			
Bucket Versioning Versioning is a means of keeping multiple variants of an object in the sa easily recover from both unintended user actions and application failure Bucket Versioning Enabled			

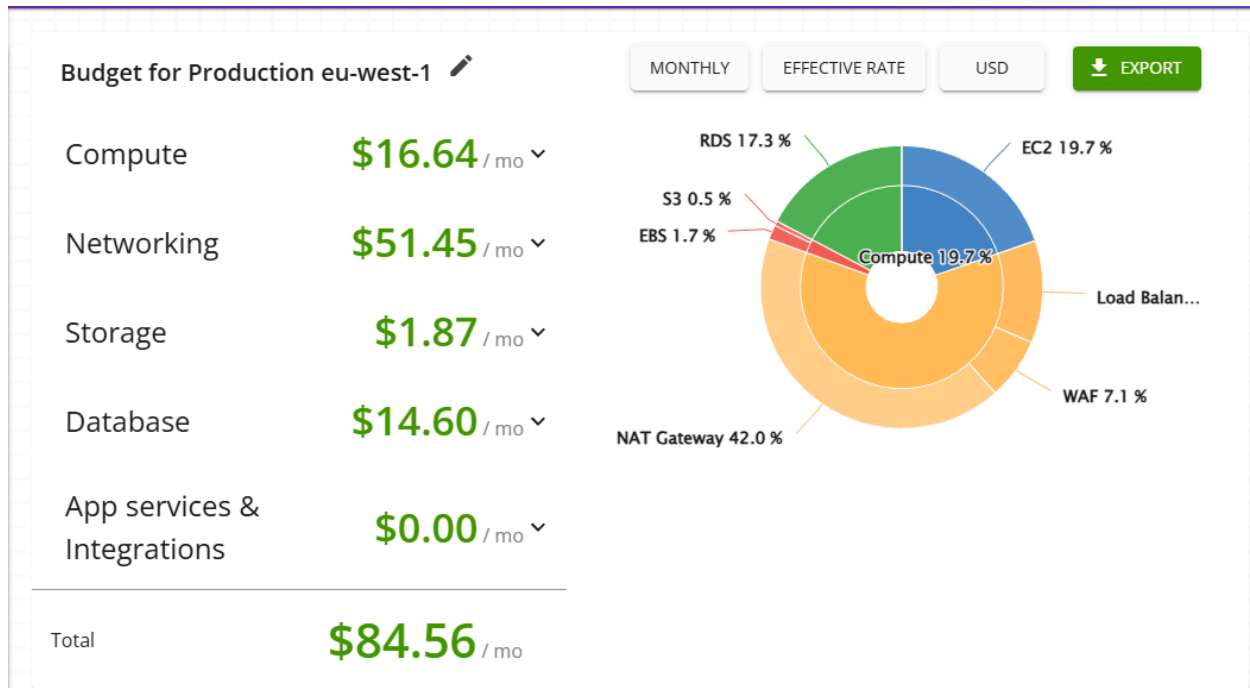
Phase 1-B : Architecture et gestion des coûts

Voici notre diagramme généré automatiquement par CloudCraft.:



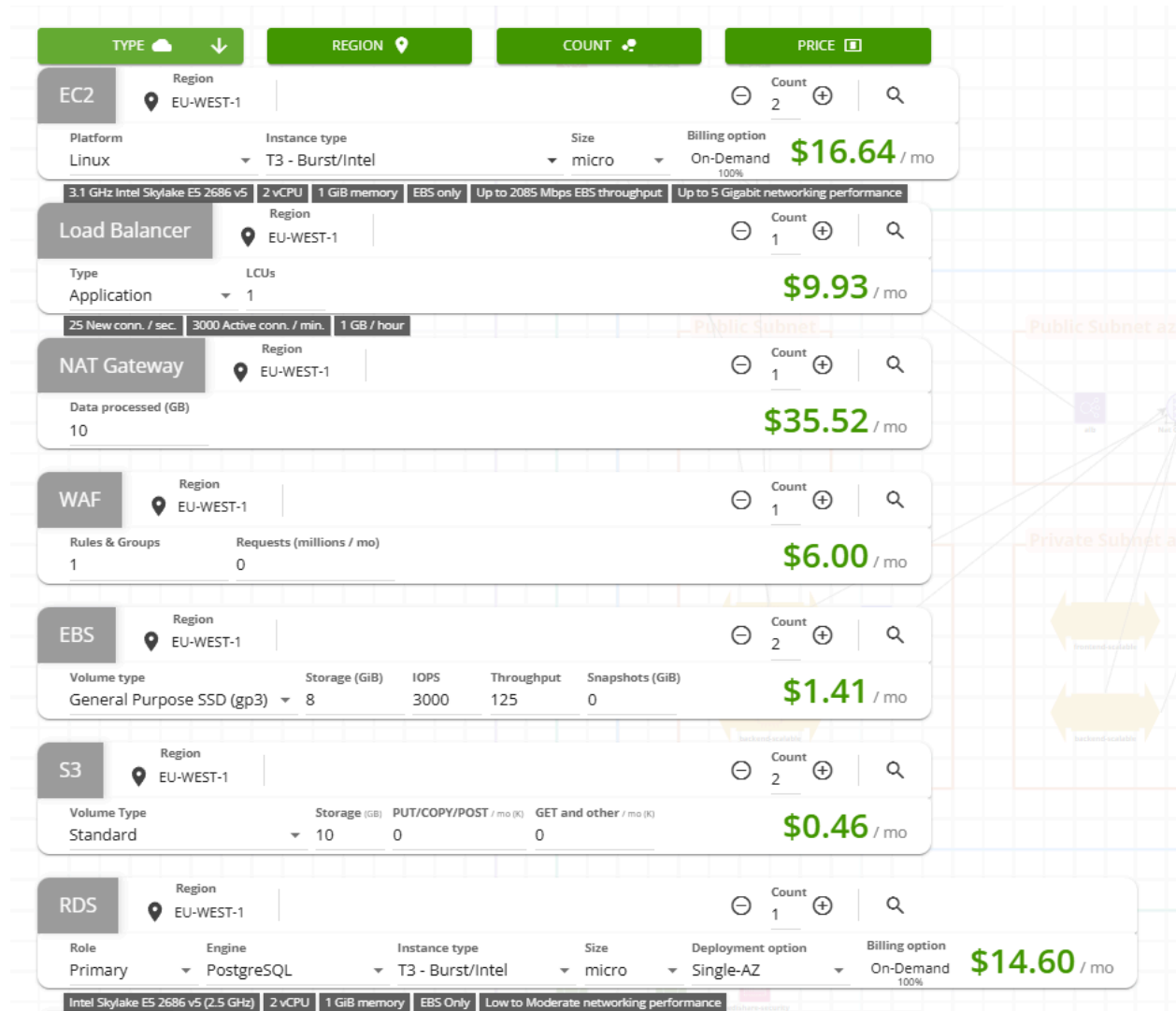
Nous avons bien une segmentation public/private subnet (best practice) et l'architecture est multi-az pour la haute disponibilité. Les services hors vpc tels que S3,WAF sont renseigné à l'intérieur du carré violet. On voit bien notre auto scaling group avec les flèches horizontales indiquant qu'il s'agit d'un scaling horizontal. Enfin, on constate également les volumes EBS chiffrés de nos machines EC2, et également tout en bas l'alerte.

Concernant les coûts, voici les estimations:



On remarque que la Nat Gateway est ce qui coûte le plus cher; ceci est cohérent car elle facturée pour chaque heure d'utilisation. En deuxième nous avons nos machines EC2 qui sont en T3.micro. Ensuite nous trouvons RDS, qui est multi-AZ avec 10go de stockage. Ensuite nos applications load balancer (multi az), qui sont facturés à l'heure également. Enfin S3 et EBS ne coûtent pas cher.

Plus en détail on trouve ceci:



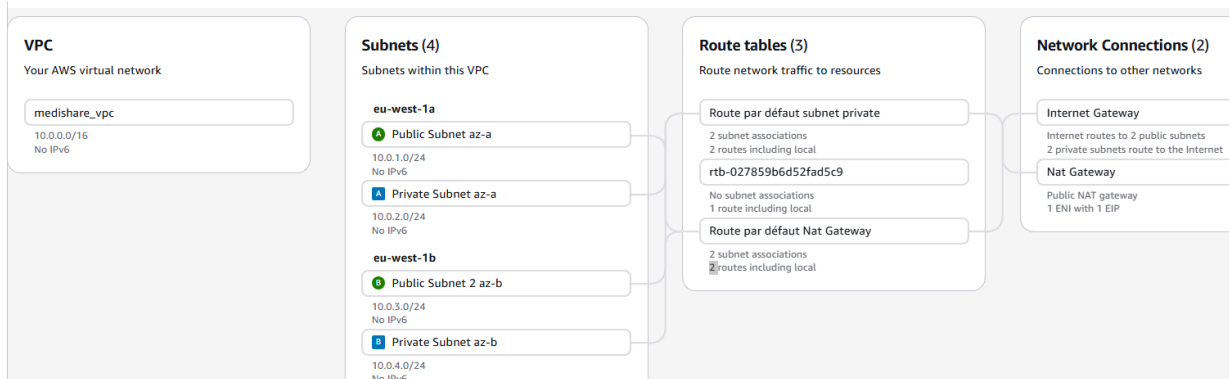
Ces estimations sont cohérentes mais elles restent toujours à optimiser. On sait que le coût d'une infrastructure AWS peut être parfois divisé par 2 avec les bonnes pratiques (bons types d'instance, suppression des ressources inutiles, instances spots ,etc.)

Phase 2 : Mise en œuvre de la sécurité dans les environnements AWS

Ce projet a entièrement été réalisé dans la région Eu-west-1 (Ireland).

Il est réalisé dans le vpc medishare_vpc

vpc-064ee55f0cf996eaa / medishare_vpc



Dans le cadre de ce projet, nous avons segmenté notre réseau de la manière suivante:

- 1 Internet Gateway qui est notre passerelle pour accéder à internet.
- 1 subnets public pour chaque AZ (eu-west-1a & 1b) contenant notre application load balancer, et la nat gateway.
- 1 subnet privé pour chaque AZ (eu-west-1a & 1b) contenant nos instances ec2 et la machine RDS.

Nous avons implémenté une Nat Gateway dans un subnet public afin de ne pas exposer en public nos instances ec2 lorsqu'elle souhaite par exemple faire un `sudo apt install`. Le trafic à destination de la nat gateway est automatiquement redirigé vers l'internet gateway:

```
resource "aws_route_table" "public_subnet" {  
  
    vpc_id = aws_vpc.medishare_vpc.id  
  
    route {  
        cidr_block = "0.0.0.0/0"  
        gateway_id = aws_internet_gateway.internet_gateway.id  
    }  
  
    tags = {  
        Name = "Route par défaut Nat Gateway"  
    }  
}
```

NAT gateways (1/1) [Info](#)

Find NAT gateways by attribute or tag

Name	NAT gateway ID	Connectivity...	State	State message	Availability ...
Nat Gateway	nat-00a2a44bf0371255d	Public	Available	-	Zonal

nat-00a2a44bf0371255d / Nat Gateway

NAT gateway ID nat-00a2a44bf0371255d	Connectivity type Public	State Available	State message -
NAT gateway ARN arn:aws:ec2:eu-west-1:371447792143:natgateway/nat-00a2a44bf0371255d	Primary public IPv4 address 34.251.22.183	Primary private IPv4 address 10.0.1.74	Primary network interface ID eni-0d13eef9a8c8ea543
	Subnet	Created	Deleted

Et donc la route par défaut du subnet privé est à destination de la Nat Gateway:

```
resource "aws_route_table" "private_subnet" {

  vpc_id = aws_vpc.medishare_vpc.id

  route {

    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat_gateway.id
  }

  tags = {
    Name = "Route par défaut subnet private"
  }
}
```

Le déploiement multi AZ garantit une haute disponibilité.

Concernant les security groups:

Le frontend écoute sur le port 80/443, donc la règle entrante doit être d'autoriser ce port. Et en règle sortante on autorise le trafic à destination de tout, afin que le serveur puisse installer ses

logiciels requis.

```
resource "aws_vpc_security_group_ingress_rule" "sg_frontend-ingress_http" {
  security_group_id = aws_security_group.sg_frontend.id
  referenced_security_group_id = aws_security_group.sg_alb.id
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}

resource "aws_vpc_security_group_egress_rule" "sg_frontend-egress_all" {
  security_group_id = aws_security_group.sg_frontend.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1" # tous les protocoles
}
```

Pour le backend (nodejs), il est sur le port 3000, on en entrée on saisit alors le port 3000 et en sortie, même chose que le frontend:

```
resource "aws_vpc_security_group_ingress_rule" "sg_backend-ingress_nodejs" {
  security_group_id = aws_security_group.sg_backend.id
  referenced_security_group_id = aws_security_group.sg_alb.id
  from_port         = 3000
  ip_protocol       = "tcp"
  to_port           = 3000
}

resource "aws_vpc_security_group_egress_rule" "sg_backend-egress_all" {
  security_group_id = aws_security_group.sg_backend.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1"
}
```

Concernant le load balancer il s'agit d'un ALB avec 2 target group, un pour le frontend et un pour le backend.

Lorsqu'une requête cible le chemin /api/ , le trafic est redirigé vers le backend. Autrement c'est le frontend.

Listener rules (2) Info					Rule limits
Traffic received by the listener is routed according to the default action and any additional rules. Rules are evaluated in priority order from the lowest value to the highest value.					
Filter rules					
☐	Priority	Name tag	Conditions (If)	Transforms	Actions (Then)
☐	100	-	Path (value) = /api/*	-	Forward to target group tg-backend: 1 (100%) Target group stickiness: Off
☐	Last (default)	Default	If no other rule applies	-	Forward to target group tg-frontend: 1 (100%) Target group stickiness: Off

L'ALB écoute au port 80:

Listeners and rules (1) [Info](#)

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is rou

Filter listeners			
☐	Protocol:Port	Default action	Rules
☐	HTTP:80	• Forward to target group tg-frontend ↗: 1 (100%) Target group stickiness: Off	2 rules

Il peut être judicieux d'activer le HTTPS pour une communication chiffrée au port 443.

Target groups (1/2) [Info](#) | [What's new?](#)

Filter target groups				
☒	tg-backend	☐ arn:aws:elasticloadbalancin...	3000	HTTP
☐	tg-frontend	☐ arn:aws:elasticloadbalancin...	80	HTTP

Target group: tg-backend

Details	Targets	Monitoring	Health checks	Attributes	Tags
Details					
☐ arn:aws:elasticloadbalancing:eu-west-1:371447792143:targetgroup/tg-backend/6c4438688d6a423a					
Target type Instance			Protocol : Port HTTP: 3000		
IP address type IPv4			Load balancer alb ↗		
1	⊙ 1	⊗ 0			
Total targets	Healthy	Unhealthy			

On a également défini une network acl pour une meilleure sécurité réseau:

acl-0ab9d32e3dee18c70

Subnet associations (4)

Filter subnet associations

Name	Subnet ID	Associated with	Availability Zone	IPv4 CIDR
Private Subnet az-b	subnet-06584d70b5c81a35f	acl-0ab9d32e3dee18c70	euw1-az2 (eu-west-1b)	10.0.4.0/24
Public Subnet az-a	subnet-0a1f5f753221d390b	acl-0ab9d32e3dee18c70	euw1-az1 (eu-west-1a)	10.0.1.0/24
Public Subnet 2 az-b	subnet-0455cde7b2602ba77	acl-0ab9d32e3dee18c70	euw1-az2 (eu-west-1b)	10.0.3.0/24
Private Subnet az-a	subnet-0e25f241c9a2533a7	acl-0ab9d32e3dee18c70	euw1-az1 (eu-west-1a)	10.0.2.0/24

Concernant la scalabilité des machines ec2, nous avons implémenté un auto-scaling group:

Auto Scaling groups (2) Info

Last updated less than a minute ago

Launch configurations Launch templates Actions Create Auto Scaling group

Search your Auto Scaling groups

Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max	Availability Zones	Creation time
frontend-asg	frontend-template-20260325122242313	1	-	1	1	4	2 Availability Zones	Wed Mar 25 2026 1...
backend-asg	backend-template-20260325121816255	1	-	1	1	4	2 Availability Zones	Wed Mar 25 2026 1...

La capacité désirée est d'1, le minimum d'1, et on peut étendre jusqu'à 4 machines selon le trafic. Le tout est bien sûr multi-AZ pour la haute disponibilité.

Pour accéder aux machines, nous avons décidé d'utiliser SSM. SSM permet qu'un utilisateur aws avec les permissions puisse se connecter directement à une machine ec2 dans un subnet privé, de manière sécurisée. Ceci est une alternative à setup un environnement bastion qui créerait une surface d'attaque encore plus importante, notamment car il n'y a pas besoin d'ouvrir le SSH.

EC2 > Instances > [i-07c546cf7b2a41d61](#) > Connect to instance

Connect Info

Connect to an instance using the browser-based client.

Instance ID i-07c546cf7b2a41d61 (backend-scalable)	VPC ID vpc-064ee55f0cf996eaa	Security groups sg-00c5964f66cd1809b (sg_backend)	IAM role ssm-ec2-role
---	---	--	--

EC2 Instance Connect SSM Session Manager SSH client EC2 serial console

SSM agent status

Introducing Systems Manager just-in-time node access
Move towards zero standing privileges by requiring operators to request access before remotely connecting to instances. [Learn more](#)

Try for free

Ping status
Online

Last ping time
Thu Mar 26 2026 06:26:52 GMT+0100 (heure normale d'Europe centrale)

Session Manager connection status
Connected

Agent version
3.3.4108.0

Session Manager usage:

- Connect to your instance without SSH keys, a bastion host, or opening any inbound ports.
- Sessions are secured using an AWS Key Management Service key.
- You can log session commands and details in an Amazon S3 bucket or CloudWatch Logs log group.
- Configure sessions on the Session Manager [Preferences](#) page.

Cancel Connect

eu-west-1.console.aws.amazon.com/systems-manager/session-manager/i-07c546cf7b2a41d61?region=eu-west-1#

Session ID: root-7pqb3nteleka22v993p5jkdte Shortcuts Instance ID: i-07c546cf7b2a41d61

```
$ bash
ssm-user@ip-10-0-4-247:/usr/bin$
```

Phase 3 : Gestion des clés (KMS) et Secrets Manager

Pour chiffrer, nous avons utilisé KMS:

Customer managed keys (2)	
Filter keys by properties or tags	
☐ Aliases ▾	☐ Key ID ▾
☐ medishare-s3-key	2c8144a0-6b61-4cd6-b2ef-a14...
☐ medishare-kms	72077448-f77a-444e-831c-369...

La première clé sert à chiffrer les fichiers du bucket S3. La deuxième est utilisée par AWS Secrets Manager. Elle sert à chiffrer la valeur du secret medishare-db-credentials.

Concernant le secret manager:

medishare-db-credentials-6e0r

Secret details

Encryption key

 aws/secretsmanager

Secret name

 medishare-db-credentials-6e0r

Secret ARN

 arn:aws:secretsmanager:eu-west-1:371447792143:secret:medishare-db-credentials-6e0r-QEqrLo

Overview

Rotation

Versions

Replication

Tags

Secret value [Info](#)

Retrieve and view the secret value.

Key/value

Plaintext

Secret key

Secret value

engine

 postgres

Il est notamment référencé dans le code du server:

```
try {
  const response = await secretsClient.send(new GetSecretValueCommand({ SecretId: SECRET_NAME }));
  const secret = JSON.parse(response.SecretString);

  dbPool = new Pool({
    host: secret.host, user: secret.username, password: secret.password,
    database: "medisharedb", port: 5432, ssl: { rejectUnauthorized: false }
  });
}
```

Dans l'extrait de code ci-dessus, le serveur tente une connexion à la base de donnée avec la constance secret. Il y a notamment comme clés dans notre secret manager:

Secret key

engine

host

password

username

Engine = le moteur de la DB

Host = comment joindre notre DB

Password = le mot de passe du compte

Username = le nom d'utilisateur utilisé pour administrer la DB

Phase 4 : Surveillance et conformité

Nous avons implémenté Cloudwatch pour récolter des logs sur notre infrastructure.

Log groups (2)

By default, we only load up to 10,000 log groups.

🔍 *Filter log groups or try pattern search*

☐

| **Log group**

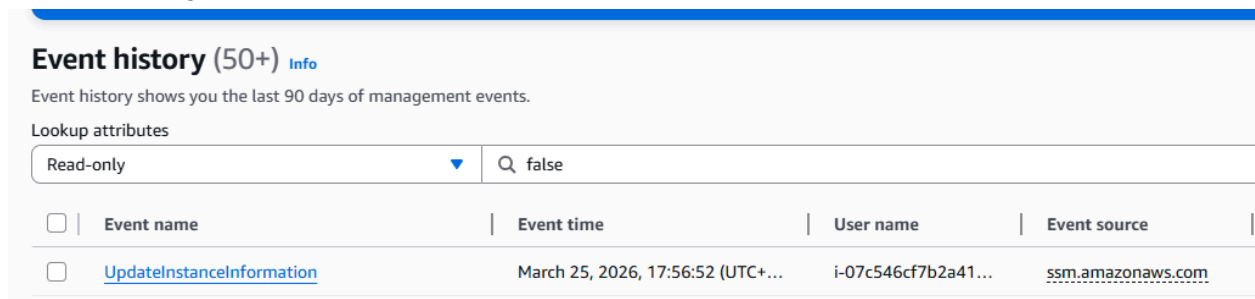
☐

[/aws/codebuild/medishare-security-sast](#)

☐

[/aws/ec2/medishare-backend](#)

Nous avons également implémenté cloudtrail pour les dernières évènements API.



Event history (50+) [Info](#)

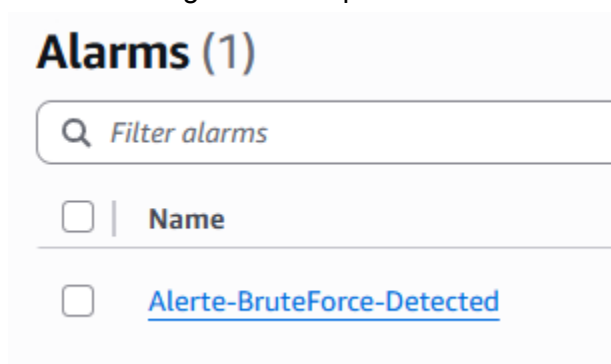
Event history shows you the last 90 days of management events.

Lookup attributes

Read-only ▼ Q false

☐	Event name	Event time	User name	Event source
☐	UpdateInstanceInformation	March 25, 2026, 17:56:52 (UTC+...	i-07c546cf7b2a41...	ssm.amazonaws.com

Nous avons également implémenté une alarme cloudwatch:

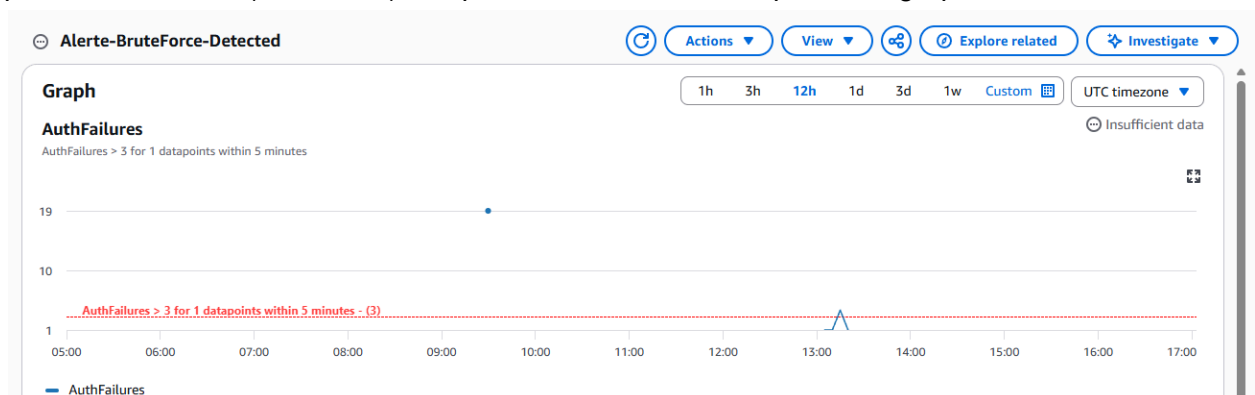


Alarms (1)

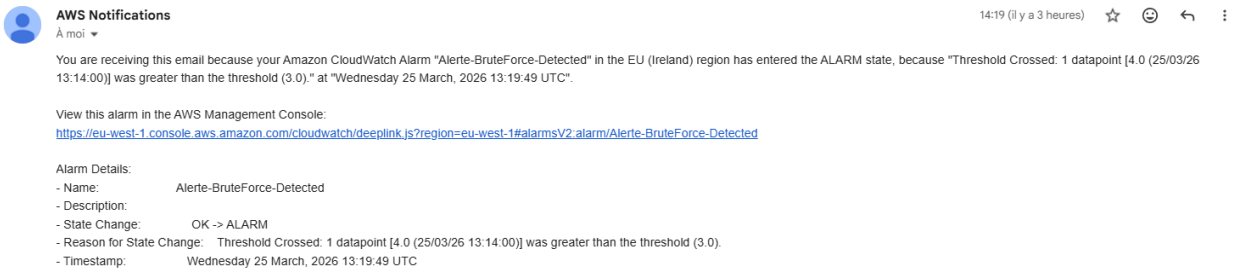
Q Filter alarms

☐	Name
☐	Alerte-BruteForce-Detected

Cette dernière indique lorsqu'un utilisateur effectue un nombre élevé de requêtes successives pour se connecter (bruteforce). On peut le constater via un pic sur le graphe:



On a configuré cela pour être alerté par mail:



Phase 5 : Application de DevSecOps

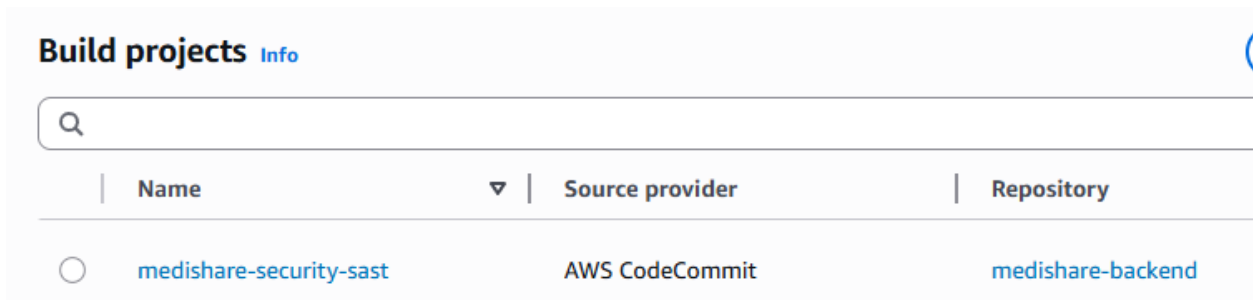
Dans le cadre de ce projet, nous avons utilisé Terraform:

```
etudiant@debian:~/projet_cloud_security/cloud_security$ ls
alb.tf      data.tf      igw.tf      monitoring.tf  readme.md  terraform.tfstate  vpc.tf
backend     frontend     init_backend.sh  nat.tf      routes.tf  terraform.tfstate.backup  waf.tf
backend.tf  frontend.tf  init_frontend.sh  providers.tf  s3.tf     terraform.tfvars
cicd.tf     iam.tf       main.tf       rds.tf      sg.tf     variables.tf
```

Ainsi, avant chaque déploiement d'un service nous faisons un terraform plan puis terraform apply. Nous faisons cela progressivement, puis vérifions que c'était correctement déployé. Ceci explique le grand nombre de fichiers .tf .

Pour éviter les déploiements manuels (sources d'erreurs humaines), nous avons mis en place une automatisation où chaque modification du code est testée. AWS CodeBuild permet de compiler l'application nodejs dans un environnement isolé, tandis que CodePipeline orchestre le passage du code de GitHub (ou CodeCommit) vers les instances EC2 via le Load Balancer.

On utilise CodeBuild pour tester le code à upload.



Pour ce faire, via terraform on lance un conteneur avec une image qui contient le package npm audit qui sert à auditer la sécurité:

<https://docs.npmjs.com/cli/v8/commands/npm-audit>

```

pre_build:
  commands:
    - echo "Démarrage de l'analyse DevSecOps"
    - npm install
build:
  commands:
    - echo "Scan des vulnérabilités avec npm audit..."
    # La commande échouera et bloquera la pipeline si une faille 'high' est trouvée
    - npm audit --audit-level=high
    - echo "✅ Scan terminé avec succès. Aucune vulnérabilité critique."
EOF
}

```

Nous avons upload ici volontairement un fichier package.json avec des dépendances anciennes vulnérables:

```

{
  "dependencies": {
    "@aws-sdk/client-s3": "^3.1016.0",
    "@aws-sdk/client-secrets-manager": "^3.1016.0",
    "@aws-sdk/s3-request-presigner": "^3.1016.0",
    "cors": "^2.8.6",
    "express": "^5.2.1",
    "helmet": "^8.1.0",
    "pg": "^8.20.0",
    "winston": "^3.19.0",
    "winston-cloudwatch": "^6.3.0"
  }
}

```

Ainsi le build nous indique 16 vulnérabilités high et 1 critique:

```

437 Depends on vulnerable versions of @aws-sdk/core
438 node_modules/@aws-sdk/credential-provider-process
439 @aws-sdk/credential-provider-sso 3.894.0 - 3.972.0
440 Depends on vulnerable versions of @aws-sdk/client-sso
441 Depends on vulnerable versions of @aws-sdk/core
442 Depends on vulnerable versions of @aws-sdk/token-providers
443 node_modules/@aws-sdk/credential-provider-sso
444 @aws-sdk/credential-provider-web-identity 3.894.0 - 3.972.0
445 Depends on vulnerable versions of @aws-sdk/core
446 Depends on vulnerable versions of @aws-sdk/nested-clients
447 node_modules/@aws-sdk/credential-provider-web-identity
448 @aws-sdk/middleware-user-agent 3.894.0 - 3.972.0
449 Depends on vulnerable versions of @aws-sdk/core
450 node_modules/@aws-sdk/middleware-user-agent
451 @aws-sdk/util-user-agent-node 3.894.0 - 3.972.0
452 Depends on vulnerable versions of @aws-sdk/middleware-user-agent
453 node_modules/@aws-sdk/util-user-agent-node
454 @aws-sdk/nested-clients 3.894.0 - 3.972.0
455 Depends on vulnerable versions of @aws-sdk/core
456 Depends on vulnerable versions of @aws-sdk/middleware-user-agent
457 Depends on vulnerable versions of @aws-sdk/util-user-agent-node
458 node_modules/@aws-sdk/nested-clients
459 @aws-sdk/token-providers 3.894.0 - 3.972.0
460 Depends on vulnerable versions of @aws-sdk/core
461 Depends on vulnerable versions of @aws-sdk/nested-clients
462 node_modules/@aws-sdk/token-providers
463
464 17 vulnerabilities (16 high, 1 critical)

```

Comme NPM audit a identifié une faille, alors le build est considéré comme failed:

Build logs > Phase details > Reports > Environment variables > Build details > Resource utilization						
Name	Status	Context	Duration	Start time	End time	
SUBMITTED	✓ Succeeded	-	<1 sec	Mar 25, 2026 12:44 PM (UTC+1:00)	Mar 25, 2026 12:44 PM (UTC+1:00)	
QUEUED	✓ Succeeded	-	<1 sec	Mar 25, 2026 12:44 PM (UTC+1:00)	Mar 25, 2026 12:44 PM (UTC+1:00)	
PROVISIONING	✓ Succeeded	-	3 secs	Mar 25, 2026 12:44 PM (UTC+1:00)	Mar 25, 2026 12:44 PM (UTC+1:00)	
DOWNLOAD_SOURCE	✓ Succeeded	-	8 secs	Mar 25, 2026 12:44 PM (UTC+1:00)	Mar 25, 2026 12:44 PM (UTC+1:00)	
INSTALL	✓ Succeeded	-	<1 sec	Mar 25, 2026 12:44 PM (UTC+1:00)	Mar 25, 2026 12:44 PM (UTC+1:00)	
PRE_BUILD	✓ Succeeded	-	36 secs	Mar 25, 2026 12:44 PM (UTC+1:00)	Mar 25, 2026 12:45 PM (UTC+1:00)	
BUILD	✗ Failed	COMMAND_EXECUTION_ERROR: Error while executing command: npm audit --audit-level=high. Reason: exit status 1	2 secs	Mar 25, 2026 12:45 PM (UTC+1:00)	Mar 25, 2026 12:45 PM (UTC+1:00)	
POST_BUILD	✓ Succeeded	-	<1 sec	Mar 25, 2026 12:45 PM (UTC+1:00)	Mar 25, 2026 12:45 PM (UTC+1:00)	
UPLOAD_ARTIFACTS	✓ Succeeded	-	<1 sec	Mar 25, 2026 12:45 PM (UTC+1:00)	Mar 25, 2026 12:45 PM (UTC+1:00)	
FINALIZING	✓ Succeeded	-	<1 sec	Mar 25, 2026 12:45 PM (UTC+1:00)	Mar 25, 2026 12:45 PM (UTC+1:00)	
COMPLETED	✓ Succeeded	-	-	Mar 25, 2026 12:45 PM (UTC+1:00)	-	

En revanche, tout le reste est un succès.

Pour garantir que l'infrastructure reste sécurisée dans le temps, on utilise AWS Config. Cet outil surveille les changements de configuration (ex: si quelqu'un ouvre un port par erreur dans

un Security Group) et génère des alertes en cas de non-conformité avec les "best practices" (ex: "S3 buckets should not be public").

Par exemple, tous nos volumes sont compliant:

Resources (8)

View details

Resource category

AWS resources

AWS EC2 Volume

Resource type

Multiple selected

Compliance

Compliant

Resource identifier - optional

Q Enter resource identifier

Include deleted resources

Apply

Clear filters

Resource identifier	Type	Compliance
☐ vol-03b02ff5ec7f07095	EC2 Volume	☒ Compliant
☐ vol-0590e188edecbba5c	EC2 Volume	☒ Compliant
☐ vol-067fa0d831e1722dd	EC2 Volume	☒ Compliant
☐ vol-077e00dadac77e5b2	EC2 Volume	☒ Compliant
☐ vol-0af8ac677d4e1e7c6	EC2 Volume	☒ Compliant
☐ vol-0c295cdd28f02905f	EC2 Volume	☒ Compliant
☐ vol-0d057a81aa790117c	EC2 Volume	☒ Compliant
☐ vol-0f092d97b55d34541	EC2 Volume	☒ Compliant

On a défini la règle suivante:

ebs-optimized-instances-encrypted			
Details			
Description Secure EBS volumes attached to EC2 instances through encryption.			
Domain Data protection	Control name Enable encryption for EBS volumes attached to EC2 instances	Behavior Detective ⓘ	
Objectives Data encryption	Service Amazon Elastic Compute Cloud (Amazon EC2)	Aliases AWS-GR_ENCRYPTED_VOLUMES	
Common controls Encrypt data at rest	Governed resources AWS::EC2::Volume	Deployable Regions 30 of 34 Regions ⓘ	
Frameworks CCCS-Medium-Cloud-Control-May-2019 ⓘ NIST-SP-800-53-r5 ⓘ NIST-CSF-v1.1 ⓘ FedRAMP-r4 ⓘ AWS-WAF-v10 ⓘ PCI-DSS-v4.0 ⓘ PCI-DSS-v3.2.1 ⓘ ISO-IEC-27001:2013-Annex-A ⓘ SSAE-18-SOC-2-Oct-2023 ⓘ CIS-AWS-Benchmark-v1.4 ⓘ CIS-AWS-Benchmark-v1.3 ⓘ ACSC-ISM-02-Mar-2023 ⓘ NIST-SP-800-171-r2 ⓘ CIS-v7.1 ⓘ CIS-v8.0 ⓘ	API identifier arn:aws:controlcatalog::control/503uicglhjokaajywfpt6ros		
Scope of changes All changes	Last evaluation status Successful	Trigger type Configuration changes	
Remediation action Not set	Last successful evaluation time March 25, 2026 1:38 PM	AWS Config resource types -	

Celle-ci vérifie que tous les volumes EBS sont bien chiffrés. Le comportement (behavior) detective indique que la règle ne va pas bloquer la création d'un volume non chiffré, mais elle va le signaler comme "Non-compliant" (non conforme) dans les rapports de sécurité.

Dans notre approche DevSecOps, nous utilisons AWS Config comme garde-fou. Si une instance est lancée sans chiffrement EBS ou si un bucket S3 devient public par erreur, AWS Config le détecte immédiatement. C'est ce qu'on appelle la Remédiation Automatique ou la surveillance de conformité en temps réel.

Recherche : Attaques adverses en cybersécurité **des plateformes de Gestion de Contenus** **Numériques**

Les plateformes de gestion de contenus numériques , particulièrement dans le domaine de la santé comme MediShare, centralisent des informations d'une grande criticité. Face à l'évolution des cybermenaces et à l'évolution de l'Intelligence Artificielle, il est important de savoir s'en prémunir.

1. Identification et analyse des types d'attaques

A). Manipulation de contenus et Attaques IA

Deepfakes Médicaux : Avec l'émergence de l'IA générative , un attaquant peut altérer des contenus numériques de manière imperceptible. Dans le cadre de MediShare, cela pourrait se traduire par l'ajout ou la suppression de marqueurs de maladies sur une radiographie (falsification d'imagerie), ou la création de fausses ordonnances pour obtenir des substances réglementées.

Empoisonnement de données : Cette attaque vise à corrompre les données stockées pour fausser l'apprentissage des futurs algorithmes d'aide au diagnostic. En injectant des données médicales erronées mais d'apparence légitime, l'attaquant dégrade silencieusement la fiabilité de la plateforme.

Inversion de modèle : Une technique où un attaquant interroge un système de recommandation ou d'analyse statistique pour ré-identifier des patients à partir de données censées être anonymisées.

B). Attaques applicatives et réseau traditionnelles

Injection de code (SQLi / XSS) : L'attaquant insère des scripts malveillants dans les champs de saisie (ex: nom du patient ou nom du fichier) pour tromper la base de données ou le navigateur des médecins. Une injection SQL réussie permettrait d'extraire, de modifier ou de supprimer la table complète des dossiers patients.

Déni de Service Distribué (DDoS) : Surcharge volontaire des serveurs (via une multitude de requêtes simultanées) dans le but d'épuiser les ressources de calcul (CPU, RAM) ou la bande passante.

2. Impacts potentiels sur la sécurité et la confidentialité

La réussite de l'une de ces attaques sur une plateforme comme MediShare entraînerait des conséquences désastreuses à plusieurs niveaux :

A) Impact sur la Confidentialité (Violation de données) :

La fuite de données de santé constitue une violation du RGPD et des normes HDS (Hébergement de Données de Santé). L'exposition des pathologies ou des identités des patients expose ces derniers à des risques de chantage, de discrimination ou d'usurpation d'identité.

B) Impact sur l'Intégrité (Mise en danger d'autrui) :

La manipulation des contenus (Deepfakes médicaux ou altération des PDF) est le risque le plus critique. Un médecin se basant sur un rapport falsifié pourrait prescrire un traitement inadapté ou mortel.

C) Impact sur la Disponibilité (Rupture de soin) :

Une attaque par déni de service (DDoS) ou par ransomware rendrait la plateforme indisponible. Les praticiens perdraient l'accès immédiat à l'historique médical de leurs patients, paralysant ainsi les consultations et les urgences.

3. Stratégies de défense et bonnes pratiques

Pour contrer ces menaces, une approche de Défense en Profondeur et de Zero Trust doit être adoptée, comme illustré dans les phases de notre projet AWS :

Contre les manipulations et attaques adverses :

A) Signatures numériques et Hachage :

- Calculer l'empreinte cryptographique (hash) de chaque document lors de son upload sur S3 pour garantir son immuabilité. Toute modification ultérieure modifiera le hash, alertant immédiatement le système de la falsification.

B) Confidentialité différentielle :

- Ajouter un "bruit" mathématique lors de l'exportation de statistiques médicales pour empêcher les attaques par inversion de modèle, tout en gardant les données utiles pour la recherche.

C) Contre les Injections de code :

- Validation stricte des entrées : refuser tout caractère non alphanumérique dans les noms de fichiers et utiliser des requêtes SQL paramétrées (comme implémenté avec la bibliothèque pg dans notre backend nodejs) pour neutraliser les injections SQL.
- Déploiement d'un AWS WAF (Web Application Firewall) : placé devant l'Application Load Balancer (ALB), il filtre le trafic malveillant et bloque les requêtes contenant des signatures d'injections connues (SQLi, XSS).

D) Contre le Déni de Service (DDoS) :

- Élasticité de l'infrastructure : utiliser des Auto Scaling Groups (ASG) capables d'absorber des pics de charge imprévus en démarrant de nouvelles instances EC2 dynamiquement.

E) Bouclier Réseau :

- Activer AWS Shield Standard pour mitiger les attaques DDoS au niveau des couches de transport (L3/L4), couplé à des limites de taux de requêtes (Rate Limiting) sur les API.

Protection globale et DevSecOps :

- Chiffrement de bout en bout : chiffrer les données au repos via AWS KMS (AES-256 sur S3 et EBS) et en transit via TLS/HTTPS.
- Moindre Privilège : restreindre les accès réseaux via des Security Groups hermétiques et utiliser des rôles IAM avec permissions limitées pour éviter le vol d'identifiants à hauts privilèges.

Conclusion

Ce projet de cloud security sur aws fut très enrichissant et nous avons pu apprendre à mettre en pratique conjointement un grand nombre de services. Le tout nous a permis de comprendre l'intérêt d'une infrastructure as code grâce à nos fichiers terraform. Nous avons pu comprendre toute la puissance du cloud AWS lorsqu'il s'agit de haute disponibilité (multi-az, auto scaling group,...) et de sécurité (roles,acl,waf,kms,...).